

Lección 27: Formularios Usables

Forms que no Frustran al Usuario

Agenda

- ¿Por qué los Formularios Importan?
 - Anatomía de un Buen Formulario
 - Mejores Prácticas de Diseño de Inputs
 - Patrones de Validación
 - Mensajes de Error
 - Optimización de Formularios Móviles
 - Autofill y Autocomplete
 - Revelación Progresiva (Progressive Disclosure)
-

El Costo de Formularios Malos (1/2)

Estadísticas:

67% de usuarios abandonan formularios por complejidad
85% abandonan si los errores son confusos
27% abandonan si el formulario es muy largo
40% abandonan si la experiencia móvil es mala

Impacto en el Negocio:

E-commerce checkout: 70% abandono de carritos
– \$18 mil millones perdidos anualmente
– Formularios malos = razón principal

Cada campo removido:

- Aumenta conversión 10-20%

Cada segundo ahorrado:

- Aumenta conversión 1-2%

El Costo de Formularios Malos (2/2)

Buenos formularios = más conversiones = más ingresos

Anatomía de un Buen Formulario

Elementos esenciales:

```
<form>
  { /* 1. Título claro */ }
  <h2>Información de Envío</h2>

  { /* 2. Inputs etiquetados */ }
  <label htmlFor="email">
    Email
    <input id="email" type="email" placeholder="tu@ejemplo.com" aria-requir
  </label>

  { /* 3. Texto de ayuda */ }
  <span className="help-text">Enviaremos confirmación del pedido aquí</span>

  { /* 4. Validación inline */ }
  {errors.email && (
    <span className="error" role="alert">
      {errors.email}
    </span>
  )}

  { /* 5. Acción clara */ }
  <button type="submit">Continuar al Pago</button>
</form>
```

Diseño de Inputs: Etiquetas (1/2)

 **Mal:**

```
// Sin label
<input type="text" placeholder="Nombre" />

// Placeholder como label (desaparece al hacer foco)
<input type="text" placeholder="Nombre Completo" />

// Label muy lejos del input
<div>
  <label>Email</label>
  <div style={{ marginTop: '50px' }}>
    <input type="email" />
  </div>
</div>
```

Diseño de Inputs: Etiquetas (2/3)

 **Bien:**

```
// Label siempre visible
<label htmlFor="name">
  Nombre Completo
  <input id="name" type="text" placeholder="Juan Pérez" />
</label>

// 0: Label arriba del input
<div>
  <label htmlFor="email">Dirección de Email</label>
  <input
    id="email"
    type="email"
    placeholder="tu@ejemplo.com"
  />
</div>
```

Diseño de Inputs: Etiquetas (3/3)

Beneficios:

- Área de click más grande
 - Siempre visible
 - Accesible para lectores de pantalla
 - Contexto claro
-

Diseño de Inputs: Tipos (1/2)

Usa tipos de input correctos:

```
// ❌ Todos text inputs
<input type="text" name="email" />
<input type="text" name="phone" />
<input type="text" name="birthdate" />
<input type="text" name="quantity" />

// ✅ Tipos semánticos
<input type="email" name="email" />
<input type="tel" name="phone" />
<input type="date" name="birthdate" />
<input type="number" min="1" max="99" name="quantity" />
```

Diseño de Inputs: Tipos (2/2)

Beneficios en móviles:

```
type="email"    → Muestra tecla @ en móvil
type="tel"      → Muestra teclado numérico
type="number"   → Muestra teclado de números
type="date"     → Muestra selector de fecha
type="url"      → Muestra tecla .com
```

Tamaño de Inputs

Ajusta el ancho del input al contenido esperado:

```
// ❌ Todos del mismo ancho
<input type="text" className="full-width" /> { /* Código postal */}
<input type="text" className="full-width" /> { /* Dirección */}

// ✅ Tamaño proporcional
<input
  type="text"
  name="zip"
  className="w-24" // CP: 5 caracteres
  maxLength={5}
/>

<input
  type="text"
  name="street"
  className="w-full" // Dirección: largo variable
/>

<input
  type="text"
  name="cardCVC"
  className="w-16" // CVC: 3-4 caracteres
  maxLength={4}
/>
```

El usuario sabe qué esperar según el ancho del input

Validación: Cuándo Validar (1/2)

❌ En onChange (demasiado agresivo):

```
<input
  onChange={e => {
    if (!isValidEmail(e.target.value)) {
      setError('Email inválido') // ¡Muestra error mientras escribe!
    }
  }}
/>
```

❌ Solo en submit (demasiado tarde):

```
<form onSubmit={(e) => {  
  if (!isValidEmail(email)) {  
    setError('Email inválido') // Usuario ya hizo click en submit  
  }  
}}>
```

Validación: Cuándo Validar (2/2)

✓ En blur + en submit:

```
<input  
  onBlur={e => {  
    // Validar cuando usuario sale del campo  
    if (e.target.value && !isValidEmail(e.target.value)) {  
      setErrors(prev => ({ ...prev, email: 'Email inválido' }))  
    }  
  }}  
  onChange={e => {  
    // Limpiar error mientras escribe (si existe error)  
    if (errors.email) {  
      setErrors(prev => ({ ...prev, email: undefined }))  
    }  
  }}  
</>
```

Validación: Inline vs Resumen (1/2)

Validación inline (preferida):

```
<div className="field">  
  <label htmlFor="email">Email</label>  
  <input  
    id="email"  
    type="email"  
    aria-invalid={!!errors.email}  
    aria-describedby={errors.email ? 'email-error' : undefined}  
  />  
  {errors.email && (  
    <span id="email-error" className="error" role="alert">
```

```
        {errors.email}  
      </span>  
    })  
  </div>
```

Validación: Inline vs Resumen (2/2)

Resumen arriba (para muchos errores):

```
{  
  Object.keys(errors).length > 0 && (  
    <Alert variant="error" role="alert">  
      <strong>Por favor corrige los siguientes errores:</strong>  
      <ul>  
        {Object.entries(errors).map(([field, message]) => (  
          <li key={field}>  
            <a href={`#${field}`}>{message}</a>  
          </li>  
        ))}  
      </ul>  
    </Alert>  
  )  
}
```

Mejor: Combinar ambos

Mensajes de Error (1/2)

✖ Malos mensajes de error:

"Entrada inválida"
→ ¿Qué es inválido?

"Error 422: Unprocessable Entity"
→ Jerga técnica

"El valor ingresado no coincide con el formato requerido"
→ ¿Qué formato?

"Incorrecto"

→ No es útil

""

→ Sin mensaje

Mensajes de Error (1/2)

✓ Buenos mensajes de error:

"Email debe incluir @"

→ Específico, accionable

"Password debe tener al menos 8 caracteres"

→ Requerimiento claro

"Este email ya está registrado. Intenta iniciar sesión."

→ Sugerencia útil

"Número de tarjeta debe tener 16 dígitos"

→ Formato esperado claro

Patrón de Mensajes de Error

Estructura de un buen mensaje de error:

1. Qué está mal
2. Por qué está mal (opcional)
3. Cómo solucionarlo

Ejemplos:

"Número de teléfono muy corto. Incluye código de área (10 dígitos)"

"Código promocional 'SUMMER' ha expirado. Navega ofertas actuales"

"Cantidad no puede exceder 99. ¿Compras al mayor? Contacta ventas"

Tono: Útil, no acusatorio

✗ "Ingresaste un email inválido"

✓ "Email debe incluir @"

✗ "No puedes dejar esto en blanco"

✓ "Nombre es requerido"

Campos Requeridos (1/3)

✗ **Mal:**

```
// Todos los campos se ven iguales
<input type="text" name="name" />
<input type="text" name="email" />
<input type="text" name="phone" />
```

Campos Requeridos (1/3)

✓ **Bien - Indicador visual:**

```
<label>
  Nombre Completo *
  <input type="text" name="name" required />
</label>

<label>
  Dirección de Email *
  <input type="email" name="email" required />
</label>

<label>
  Teléfono (opcional)
  <input type="tel" name="phone" />
</label>

{/* Leyenda */}
<p className="text-sm text-gray-600">
  * Campos requeridos
</p>
```

Campos Requeridos (1/3)

Mejor: Marcar campos opcionales en vez

```
// Si la mayoría de campos son requeridos, marcar los opcionales
<label>
  Teléfono (opcional)
  <input type="tel" name="phone" />
</label>
```

Valores Predeterminados Inteligentes

Prellenar cuando sea posible:

```
// ❌ Formulario vacío
<input type="text" name="country" />

// ✅ Valor predeterminado inteligente
<input
  type="text"
  name="country"
  defaultValue={getUserCountry()} // "Argentina" para usuarios AR
/>

// ¿Envío igual que facturación?
<Checkbox
  label="Dirección de envío igual que facturación"
  checked={sameAsBilling}
  onChange={(checked) => {
    if (checked) {
      setShipping(billing) // Copiar facturación a envío
    }
  }}
/>

// Recordar preferencias del usuario
<select name="payment" defaultValue={lastUsedPaymentMethod}>
  <option value="card">Tarjeta de Crédito</option>
  <option value="paypal">PayPal</option>
</select>
```

Ahorrar tiempo al usuario = mayor conversión

Autofill y Autocomplete

Habilitar autofill del navegador:

```
<form autoComplete="on">
  <input
    type="text"
    name="name"
    autoComplete="name" // Navegador sabe que es un nombre
  />

  <input type="email" name="email" autoComplete="email" />

  <input type="tel" name="phone" autoComplete="tel" />

  <input type="text" name="address" autoComplete="street-address" />

  <input type="text" name="city" autoComplete="address-level2" />

  <input type="text" name="state" autoComplete="address-level1" />

  <input type="text" name="zip" autoComplete="postal-code" />

  <input type="text" name="cc-number" autoComplete="cc-number" />
</form>
```

Resultado: Formulario se llena en 1 click

Campos de Password (1/2)

Mejores prácticas para passwords:

```
const PasswordField = () => {
  const [showPassword, setShowPassword] = useState(false)

  return (
    <div className="field">
      <label htmlFor="password">Password</label>
```

```

<div className="password-input">
  <input
    id="password"
    type={showPassword ? 'text' : 'password'}
    autoComplete="new-password"
    minLength={8}
  />

  <button
    type="button"
    onClick={() => setShowPassword(!showPassword)}
    aria-label={showPassword ? 'Ocultar contraseña' : 'Mostrar contra
  >
    {showPassword ? <EyeOff /> : <Eye />}
  </button>
</div>

{/* Indicador de fortaleza de contraseña */}
<PasswordStrength password={password} />

{/* Requisitos */}
<ul className="text-sm">
  <li className={password.length >= 8 ? 'valid' : 'invalid'}>Al menos
  <li className={/[A-Z]/.test(password) ? 'valid' : 'invalid'}>Una le
  <li className={/[0-9]/.test(password) ? 'valid' : 'invalid'}>Un núm
</ul>
</div>
)
}

```

Campos de Password (2/2)

No hacer:

- Deshabilitar paste (¡los usuarios usan password managers!)
 - Ocultar password por defecto sin toggle
 - Requerimientos poco claros
-

Revelación Progresiva (Progressive Disclosure)

(1/2)

Mostrar campos solo cuando se necesitan:

```
<div>
  <Radio
    label="Método de envío"
    options=[
      { value: 'standard', label: 'Estándar (5-7 días)' },
      { value: 'express', label: 'Express (2-3 días)' },
    ]
    value={shippingMethod}
    onChange={setShippingMethod}
  />

  {/* Mostrar preferencias de seguimiento solo si es express */}
  {shippingMethod === 'express' && (
    <Checkbox
      label="Enviarme actualizaciones de seguimiento por SMS"
      checked={wantsSMSTracking}
      onChange={setWantsSMSTracking}
    />
  )}

  {/* Mostrar input de teléfono solo si usuario quiere SMS */}
  {wantsSMSTracking && (
    <input type="tel" name="phone" placeholder="Teléfono para actualizacion"
  )}
</div>
```

Revelación Progresiva (Progressive Disclosure)

(2/2)

Beneficios:

- Formularios más simples
 - Menos abrumador
 - Completado más rápido
-

Formularios Multi-Paso

Cuando el formulario es largo, dividir en pasos:

```
const CheckoutSteps = () => {
  const [step, setStep] = useState(1)

  return (
    <div>
      {/* Indicador de progreso */}
      <StepIndicator currentStep={step} totalSteps={3}>
        <Step number={1} label="Envío" />
        <Step number={2} label="Pago" />
        <Step number={3} label="Revisión" />
      </StepIndicator>

      {/* Paso 1: Envío */}
      {step === 1 && (
        <ShippingForm
          onNext={data => {
            saveShipping(data)
            setStep(2)
          }}
        />
      )}

      {/* Paso 2: Pago */}
      {step === 2 && (
        <PaymentForm
          onNext={data => {
            savePayment(data)
            setStep(3)
          }}
          onBack={() => setStep(1)}
        />
      )}

      {/* Paso 3: Revisión */}
      {step === 3 && <OrderReview onSubmit={placeOrder} onBack={() => setSt
    </div>
  )
}
```

Optimización Móvil (1/2)

Desafíos de formularios móviles:

- Pantallas pequeñas
 - Objetivos táctiles
 - Teclado virtual
 - Escritura lenta
 - Problemas de autocorrección
-

Optimización Móvil (2/2)

Soluciones:

```
// Objetivos táctiles más grandes (mín 44x44px)
<button className="min-h-11 min-w-11">
  Enviar
</button>

// Tipos de input correctos (activa teclado correcto)
<input type="email" /> // Muestra tecla @
<input type="tel" />    // Muestra teclado numérico
<input type="number" /> // Muestra teclado de números

// inputmode para control fino
<input
  type="text"
  inputMode="numeric" // Teclado numérico pero permite type="text"
  pattern="[0-9]*"
/>

// Deshabilitar autocorrección para ciertos campos
<input
  type="text"
  name="username"
  autoCorrect="off"
  autoCapitalize="off"
/>

// Layout de una sola columna
<div className="flex flex-col gap-4">
  { /* Un campo por fila */ }
```

</div>

Optimización Móvil (cont.)

Evitar zoom al hacer foco (iOS):

```
/* Si input font-size < 16px, iOS hace zoom */
input {
  font-size: 16px; /* Previene zoom */
}
```

Botón de acción fijo:

```
// Mantener botón de envío visible mientras teclado está abierto
<div className="fixed bottom-0 left-0 right-0 p-4 bg-white shadow-lg">
  <button className="w-full">Continuar al Pago</button>
</div>
```

Probar en dispositivos reales:

- Android: Chrome
- iOS: Safari
- Diferentes tamaños de pantalla

Confirmación y Feedback (1/2)

Después de submit:

```
// ❌ Sin feedback
<button onClick={submitForm}>Enviar</button>

// ✅ Feedback claro
<button
  onClick={submitForm}
  disabled={isSubmitting}
>
  {isSubmitting ? (
```

```

        <>
        <Spinner /> Procesando...
    </>
    ) : (
        'Realizar Pedido'
    )}
</button>

// Estado de éxito
{isSuccess && (
    <Alert variant="success">
        <Check /> ¡Pedido realizado! Confirmación enviada a {email}
    </Alert>
)}

// Redireccionar después del éxito
useEffect(() => {
    if (isSuccess) {
        setTimeout(() => {
            navigate('/order-confirmation')
        }, 2000)
    }
}, [isSuccess])

```

Confirmación y Feedback (2/2)

Prevenir doble submit:

```

const [submitted, setSubmitted] = useState(false)

const handleSubmit = async e => {
    e.preventDefault()

    if (submitted) return // Prevenir envío doble

    setSubmitted(true)
    await submitOrder()
}

```

Checklist de Mejores Prácticas para

Formularios (1/2)

Diseño:

- ☒ Etiquetas siempre visibles
- ☒ Tipos de input correctos
- ☒ Ancho de input coincide con contenido
- ☒ Objetivos táctiles grandes (móvil)
- ☒ Layout de una sola columna

Validación:

- ☒ Validar en blur + submit
 - ☒ Mensajes de error claros
 - ☒ Errores inline cerca de campos
 - ☒ Campos requeridos marcados
-

Checklist de Mejores Prácticas para Formularios (2/2)

Usabilidad:

- ☒ Autofill habilitado
- ☒ Valores predeterminados inteligentes
- ☒ Campos opcionales marcados
- ☒ Toggle para mostrar password
- ☒ Prevenir doble submit

Accesibilidad:

- ☒ Etiquetas asociadas con inputs
 - ☒ Mensajes de error anunciados
 - ☒ Accesible por teclado
 - ☒ Foco visible
-

Ejercicio 1: Validación onBlur (Validación

Progresiva)

Prompt:

Actúa como un UX engineer implementando progressive validation pattern para

CONTEXTO: Progressive validation = validar DESPUÉS que usuario termina (NOT onChange validation = aggressive (muestra error mientras usuario todavía es onsubmit validation = too late (usuario ya invirtió esfuerzo, discover error validation = balanced (valida cuando usuario sale del field, menos intrusivo onBlur validation reduce form abandonment 15-20% vs onChange. Clear error if usuario regresa a fix (onFocus), clear error para no mostrar stale error mis Inline errors: mostrar error NEXT to field (NOT summary al top solo). aria-label para screen reader accessibility.

TAREA: Implementa validación onBlur para quantity input field en carrito.

IMPLEMENTACIÓN:

- Archivo: src/shared/hooks/useQuantityValidation.ts (o similar)
- Pattern: onBlur (validate) + onFocus (clear error)
- Validación: value debe ser 1-99 (business rules)
- Mensaje de error: "Quantity must be between 1 and 99"

PROGRESSIVE VALIDATION PATTERN:

```
export function useQuantityValidation() {
  const [error, setError] = useState<string | null>(null)

  const handleBlur = (value: number) => {
    // Validar SOLO cuando usuario sale del campo
    if (value < 1 || value > 99) {
      setError('Quantity must be between 1 and 99')
    } else {
      setError(null)
    }
  }

  const clearError = () => setError(null)

  return { error, handleBlur, clearError }
}
```

USO EN COMPONENTE:

```
const { error, handleBlur, clearError } = useQuantityValidation()
```

```

<input
  type="number"
  onBlur={(e) => handleBlur(Number(e.target.value))}
  onFocus={clearError} // Limpiar error cuando usuario empieza a corregir
  aria-invalid={!error}
  aria-describedby={error ? 'quantity-error' : undefined}
/>
{error && (
  <span id="quantity-error" role="alert" className="error">
    {error}
  </span>
)}

```

TESTING UX:

1. Focus en input + escribir valor inválido (ej., 150)
2. Verificar: NO se muestra error mientras escribe (buena UX)
3. Blur (hacer click fuera del input)
4. Verificar: Aparece error: "Quantity must be between 1 and 99"
5. Focus en input nuevamente
6. Verificar: Error desaparece inmediatamente (permite corregir)
7. Escribir valor válido (ej., 5) + blur
8. Verificar: Sin error (validación pasa)

VALIDACIÓN: onBlur timing + onFocus clear = non-intrusive validation

Aprende: Validación progresiva (onBlur)

balancea utilidad con no-intrusividad

Ejercicio 2: Mensajes de Error Útiles (Microcopy Útil)

Prompt:

Actúa como un UX writer mejorando error messages de generic a helpful + act

CONTEXT0: Generic error messages = frustration + form abandonment. UX resea users abandon forms si errors confusing. Error message structure: 1) What's 2) How to fix. Tone matters: "must be" (helpful) vs "you entered" (blaming) examples: mostrar formato esperado (e.g., "<name@example.com>"). Avoid jarg

"validation failed", SÍ "Email must include @". Error message length: keep words (scan quickly). Positive framing: "Quantity must be 1–99" better que quantity". Accessibility: error messages con role="alert" para screen reader

TAREA: Transforma error messages genéricos a específicos + accionables.

PRINCIPIOS DE MENSAJES DE ERROR:

1. Ser específico: Qué campo, qué problema
2. Ser accionable: Cómo corregir (rango válido, formato esperado)
3. Ser útil: NO culpar ("ingresaste mal"), SÍ guiar ("debe ser")
4. Mostrar ejemplos: Formato esperado visible

TRANSFORMACIONES REQUERIDAS:

```
// ❌ Genérico (no útil)
"Invalid input"
→ "Quantity must be between 1 and 99"

"Error"
→ "Email must include @"

"Field is required"
→ "Name is required to complete order"

"Wrong format"
→ "Phone format: (555) 123-4567"
```

HELPER FUNCTION PATTERN:

```
function getQuantityError(value: number, min: number, max: number) {
  if (value < min) {
    return `Quantity must be at least ${min}`
  }
  if (value > max) {
    return `Quantity cannot exceed ${max}. Buying in bulk? Contact sales`
  }
  return null
}
```

ELEMENTOS CRÍTICOS:

- Nombre de campo: "Quantity" (NO "Field")
- Restricción específica: "1 and 99" (NO "invalid")
- Accionable: "Buying in bulk? Contact sales" (alternativa si necesario)
- Tono: Útil (NO "you entered wrong value")

TESTING:

1. Enviar formulario con cantidad inválida (ej., 150)
2. Mensaje de error: "Quantity cannot exceed 99. Buying in bulk? Contact sa
3. Usuario entiende: límite es 99, alternativa es contactar ventas
4. Contraste con genérico: "Invalid input" → usuario confundido qué está ma

VALIDACIÓN: Usuario debe saber exactamente cómo corregir error después de l

Aprende: Mensajes de error específicos + accionables

reducen abandono de formularios y frustración

Puntos Clave

1. **Formularios = Ingresos:** Formularios malos cuestan \$18B anuales en carritos abandonados
2. **Cada Campo Cuenta:** Remover campos innecesarios (10-20% aumento conversión por campo)
3. **Etiquetas:** Siempre visibles, cerca del input, asociadas
4. **Tipos de Input:** Usar tipos correctos para teclados móviles
5. **Validación:** En blur + submit, no en input
6. **Mensajes de Error:** Específicos, accionables, tono útil
7. **Autofill:** Habilitar autofill del navegador (ahorra mucho tiempo)
8. **Revelación Progresiva:** Mostrar campos solo cuando se necesitan
9. **Móvil:** Objetivos más grandes, teclados correctos, evitar zoom
10. **Feedback:** Estados de carga, confirmación de éxito, prevenir envío doble